رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Machine Learning

Session 5

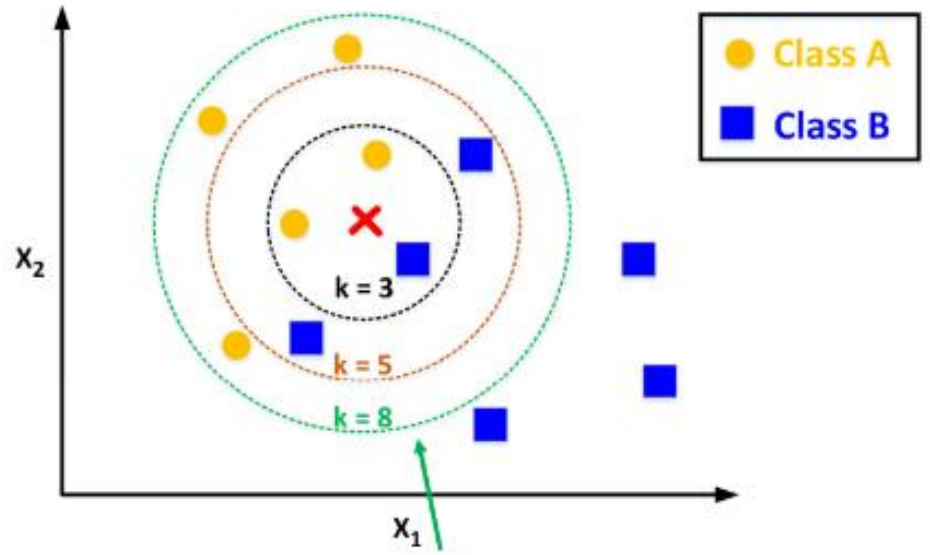
Agenda:

1	K-Nearest Neighbors (KNN)
2	Support Vector Machines (SVM)

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a **non-parametric**, **lazy learning algorithm** that is used for both **classification** and **regression**, The idea behind KNN is very **intuitive**: to classify a new data point, KNN looks at the K closest data points (neighbors) in the training set and assigns the majority class among those K neighbors to the new point.



K-Nearest Neighbors (KNN)

How KNN Works ?

1. **Choose K** (the number of neighbors).
2. **Calculate the distance** between the test point and all training points (commonly using Euclidean distance).
3. **Identify** the **K** closest points.
4. **Vote**: Classify the test point based on the majority class of its K neighbors.

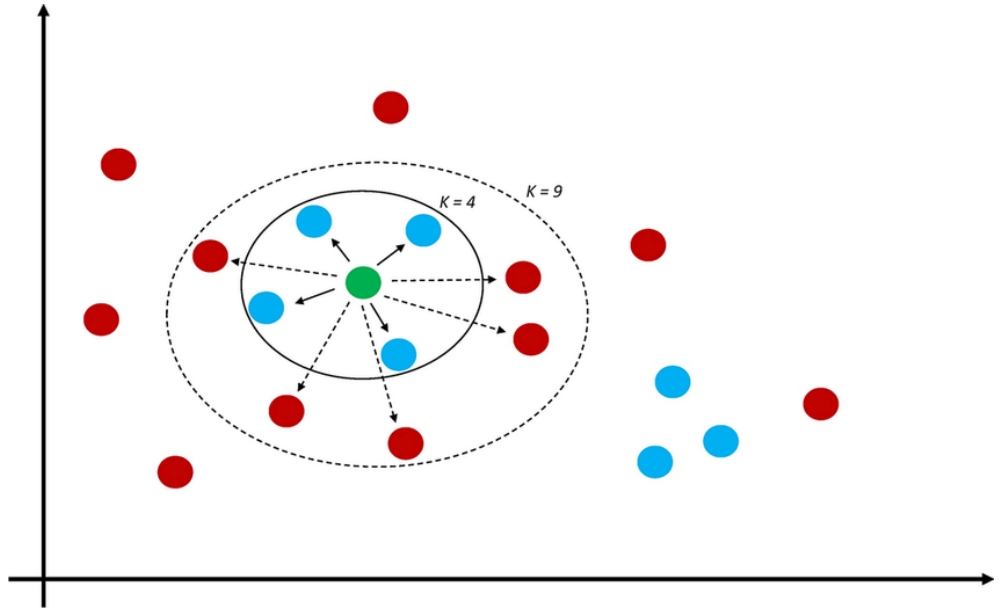
K-Nearest Neighbors (KNN)

Example of KNN

Imagine a dataset of red and blue points (two classes). We want to classify a green point using KNN (with K=4).

1. Data Points

- Red points represent Class 1.
- Blue points represent Class 2.
- A green point is a new test sample.



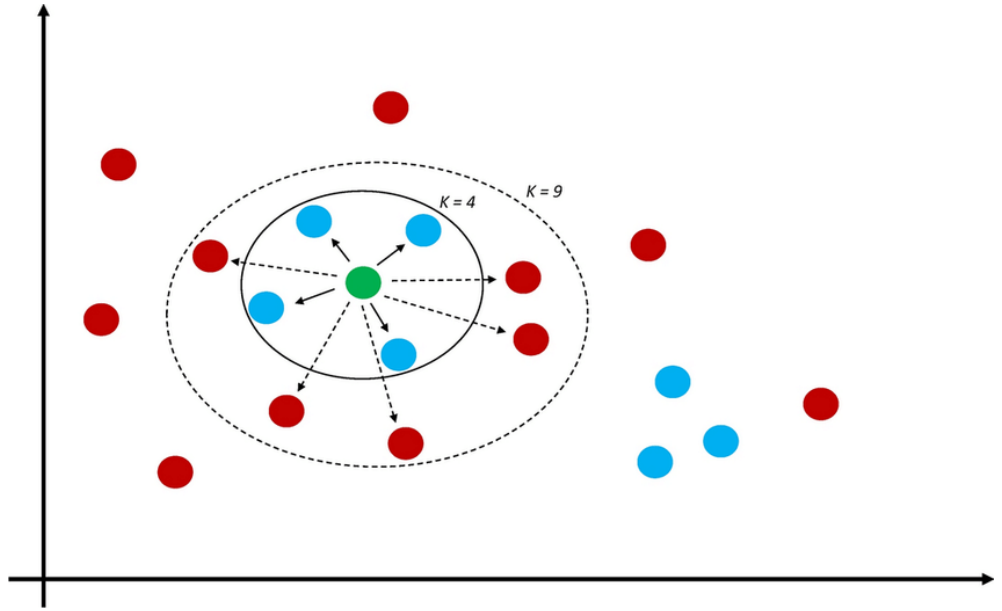
K-Nearest Neighbors (KNN)

Example of KNN

Imagine a dataset of red and blue points (two classes). We want to classify a green point using KNN (with K=4).

2. Calculate Distance

- We calculate the distance between the green point and all red/blue points.



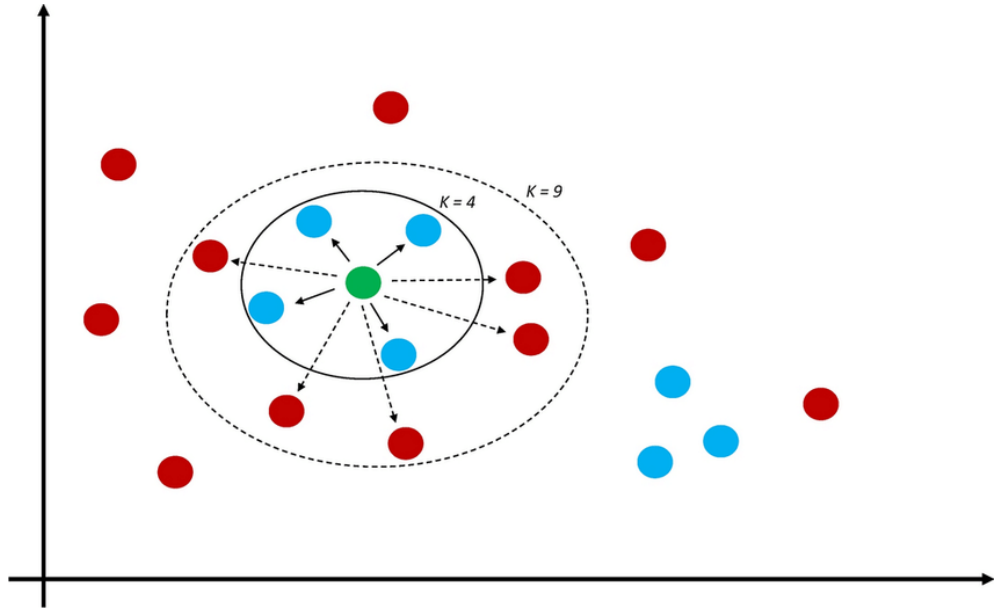
K-Nearest Neighbors (KNN)

Example of KNN

Imagine a dataset of red and blue points (two classes). We want to classify a green point using KNN (with K=4).

3. Identify K Neighbors

- Let's say K=4. We pick the 3 closest points to the green point



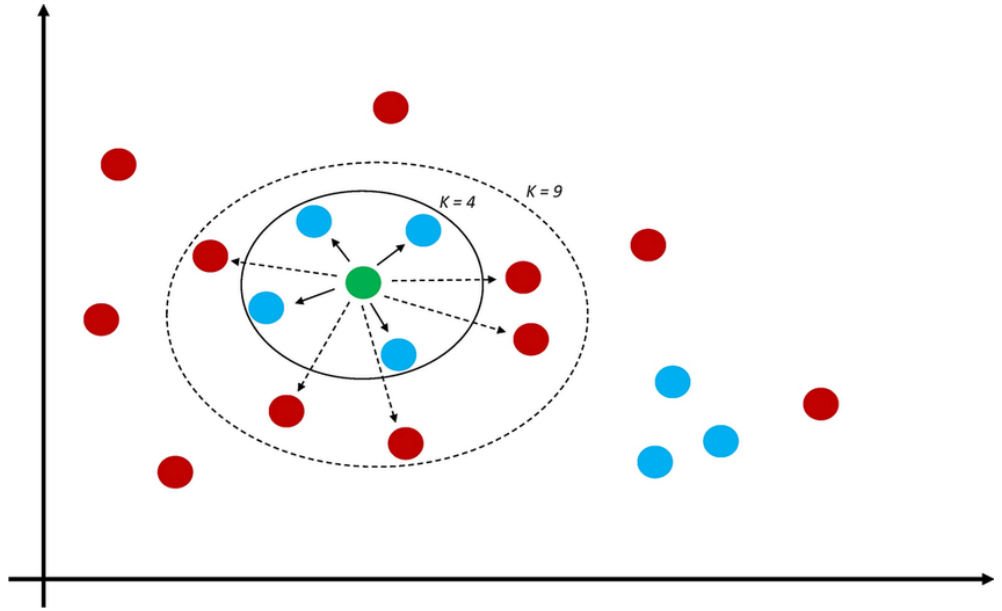
K-Nearest Neighbors (KNN)

Example of KNN

Imagine a dataset of red and blue points (two classes). We want to classify a green point using KNN (with K=4).

4. Majority Vote

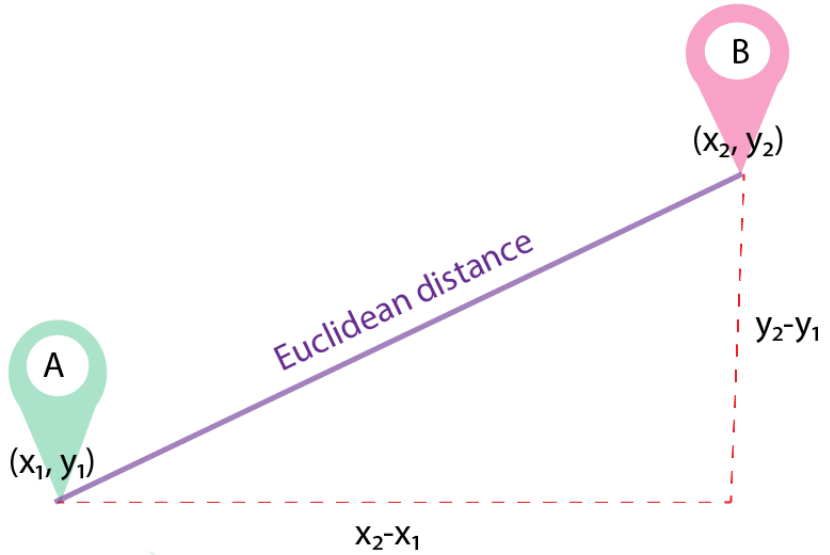
- If 2 out of 3 of the closest points are red, the green point will be classified as **Class 1** (red).



K-Nearest Neighbors (KNN)

Distance Measures for KNN

K-Nearest Neighbors (KNN) uses distance measures to determine the proximity of data points to one another. Different distance metrics affect how KNN classifies the data points.



K-Nearest Neighbors (KNN)

Distance Measures for KNN

1. Manhattan Distance

Formula $d = \sum_{i=1}^{n} |x_i - y_i|$

Explanation

- Manhattan distance is the sum of the absolute differences of their Cartesian coordinates.
- It can be visualized as "grid-like" distances, where movement is allowed only along the grid lines (no diagonal movement).

K-Nearest Neighbors (KNN)

Distance Measures for KNN

1. Manhattan Distance

Example

- If you have two points (4,4) and (1,1), the Manhattan distance is calculated as

$$d = |4 - 1| + |4 - 1| = 6$$

K-Nearest Neighbors (KNN)

Distance Measures for KNN

2. Euclidean Distance

Formula $$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Explanation

- Euclidean distance measures the straight-line distance between two points in Euclidean space.
- This is the most commonly used distance measure in KNN.

K-Nearest Neighbors (KNN)

Distance Measures for KNN

2. Euclidean Distance

Example

- Using the same points (4,4) and (1,1), Euclidean distance is calculated as

$$d = \sqrt{(4-1)^2 + (4-1)^2} = 4.24$$

K-Nearest Neighbors (KNN)

Distance Measures for KNN

3. Minkowski Distance

Formula $$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Explanation

- Minkowski distance is a generalized distance metric. It can represent both Manhattan and Euclidean distances based on the value of p:
  - When p=1, it represents Manhattan distance.
  - When p=2, it represents Euclidean distance.
- Minkowski distance is useful in vector spaces and can generalize to higher dimensions.

K-Nearest Neighbors (KNN)

Distance Measures for KNN

4. Cosine Distance/Similarity

Formula $\text{Cosine Similarity} = \cos(\theta) = \frac{a \cdot b}{|a||b|}$

Explanation

- Cosine similarity is used to measure the **similarity** between two vectors by calculating the cosine of the angle between them.
  - 1 indicates vectors pointing in the same direction.
  - 0 indicates orthogonal vectors (no similarity).
  - -1 indicates vectors pointing in opposite directions.
- Cosine **distance** is calculated as $\text{Cosine Distance} = 1 - \cos(\theta)$

Usage

- This metric is commonly used in text analysis and document comparison.

K-Nearest Neighbors (KNN)

Distance Measures for KNN

5. Hamming Distance

Formula

- Hamming distance is calculated by counting the number of bit positions in which two binary strings differ.

Explanation

- It’s used primarily for comparing two **binary data strings** of equal length.

Example

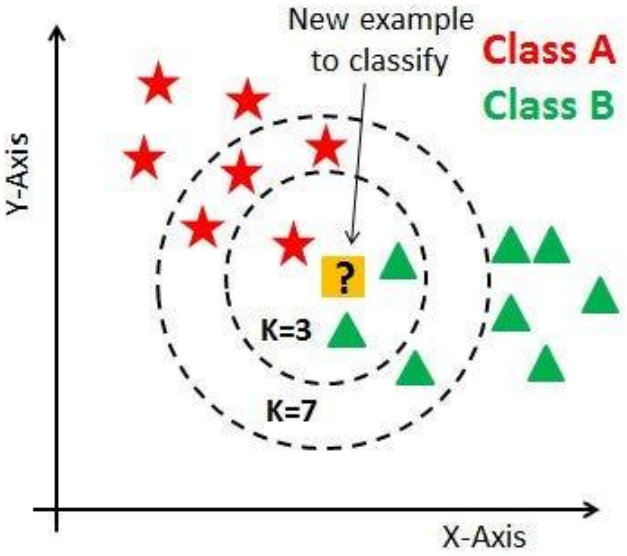
- If the strings are "ABCDE" and "AGDDF," the Hamming distance is 3 because three positions differ.

K-Nearest Neighbors (KNN)

Selecting the Best Value for k in KNN

When using **K-Nearest Neighbors (KNN)**, selecting the best value for k (the number of neighbors) is critical for achieving good classification performance.
Here are two common methods used to find the optimal k

- **Grid Search** is an exhaustive search method to try different k values.
- The **Elbow Method** is a visual technique to find the optimal k based on error or accuracy plots



K-Nearest Neighbors (KNN)

Pros and Cons of KNN

Advantages

1. **Easy to Implement**: Simple and intuitive to use.
2. **Effective for Small Datasets**: Works well with small data.
3. **Non-parametric**: No assumptions about data distribution.

K-Nearest Neighbors (KNN)

Pros and Cons of KNN

Disadvantages

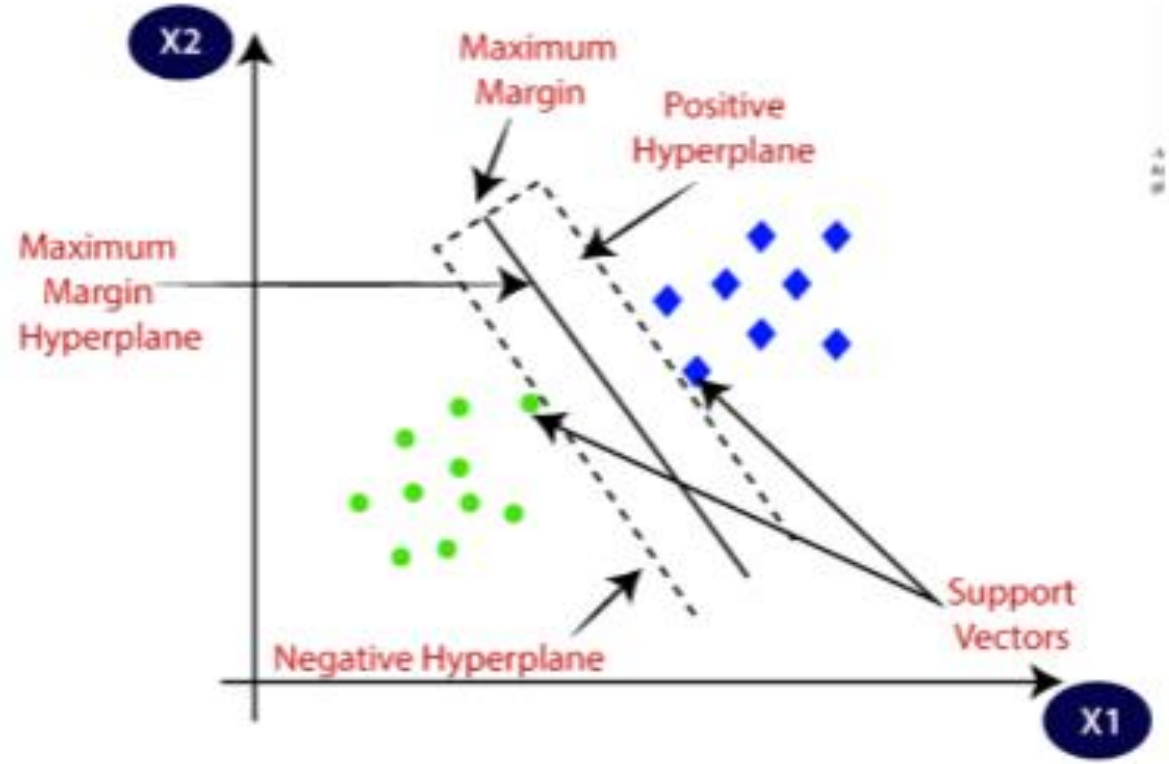
1. **Computationally Expensive**: Requires calculating the distance for every test point against all training points.
2. **Sensitive to Irrelevant Features**: KNN's performance can degrade if irrelevant or redundant features are included.
3. **Sensitive to Scaling**: Features with larger ranges will dominate unless the data is normalized.
4. **Memory Intensive**: Since KNN stores all data points, it can be memory-intensive with large datasets.
5. **Handling Missing Data**: KNN doesn't handle missing values well.
6. **Sensitive to Outliers**: Outliers can have a disproportionate influence on the classification.

Support Vector Machines (SVM)

Support Vector Machines (SVM)

Support Vector Machine (SVM) is a powerful supervised learning algorithm primarily used for classification tasks (though it can also be used for regression).
SVM works by **finding a hyperplane that best separates the data into different classes**. The goal is to find the **optimal hyperplane** with the **maximum margin** between the closest points of the two classes (called support vectors).

Support Vector Machines (SVM)



Support Vector Machines (SVM)

How SVM Works ?

1. **Linear Separation**: If the data is linearly separable, SVM finds the optimal line (in 2D) or hyperplane (in higher dimensions) that divides the classes with the maximum margin.

2. **Support Vectors**: The data points that are closest to the hyperplane are called support vectors, and they influence the position and orientation of the hyperplane.

3. **Non-Linear Data**: If the data is not linearly separable, SVM uses the kernel trick to transform the data into a higher-dimensional space where it becomes separable by a hyperplane.

Support Vector Machines (SVM)

Example of SVM

1. Linear SVM

- Imagine two classes, red and blue, that can be separated by a straight line.
- The support vectors are the points closest to the hyperplane.
- The optimal hyperplane is the one that maximizes the margin between the two classes.

Support Vector Machines (SVM)

Example of SVM

2. Non-Linear SVM with Kernel Trick

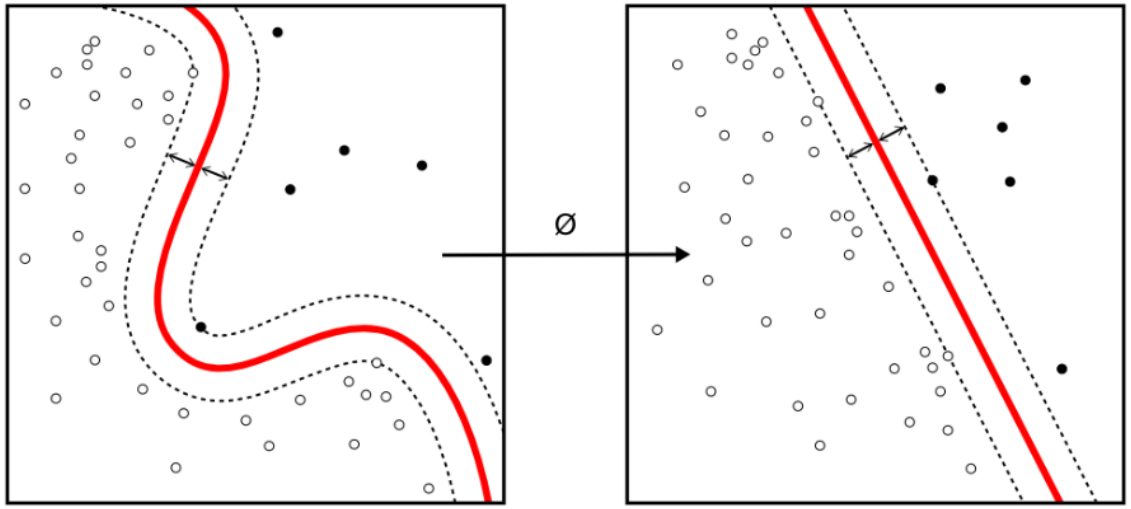
- When the data is not linearly separable, SVM applies the **kernel trick** to transform the data into a higher-dimensional space where it becomes linearly separable.

Support Vector Machines (SVM)

Example of SVM

Here's how the **kernel trick** works

- In 2D, the data might look impossible to separate with a straight line.
- By mapping the data into 3D using a kernel function, we can separate the data with a plane.



Support Vector Machines (SVM)

Advantages of SVM

- **Effective in High-Dimensional Spaces**: SVM is well-suited for problems with a large number of features.
- **Works with Both Linear and Non-Linear Data**: By applying kernel functions (e.g., radial basis function, polynomial), SVM can handle non-linear relationships.
- **Robust to Overfitting**: Especially in high-dimensional feature spaces, SVM can perform well.

Support Vector Machines (SVM)

Disadvantages of SVM

- **Computationally Expensive**: SVM can be slow for very large datasets.
- **Choosing the Right Kernel**: The performance of SVM is highly dependent on the choice of kernel, which may not always be straightforward.

KNN vs. SVM: Visual and Practical Comparison

Feature	KNN	SVM
Concept	Classifies based on the majority of K neighbors	Finds optimal hyperplane to separate classes
Model Complexity	Simple, non-parametric	Parametric, powerful for complex tasks
Distance Calculation	Based on Euclidean distance (or others)	Not based on distance but on margin maximization
Speed	Slower with large datasets	Faster prediction but slower training for large datasets
Interpretability	Simple to interpret	More complex to understand
Works Best	With smaller datasets	High-dimensional feature spaces, both linear and non-linear problems
Sensitive To	Outliers and irrelevant features	Choice of kernel and large datasets

KNN vs. SVM: Visual and Practical Comparison

Summary of Key Concepts

KNN

- Nearest Neighbors: Relies on proximity to other points.
- Majority Vote: KNN uses majority voting to classify data.
- Non-parametric: KNN does not make assumptions about the data distribution.

KNN vs. SVM: Visual and Practical Comparison

Summary of Key Concepts

SVM

- Support Vectors: Points that define the margin between classes.
- Hyperplane: SVM seeks to find the best hyperplane to separate the classes.
- Kernel Trick: A powerful tool to deal with non-linearly separable data.